



第4课-Triton基础-优化Matmul算子

第0课-OpenAI triton自制大模型推理框架项目

配套视频

第4课-triton矩阵乘法的进一步优化.mkv

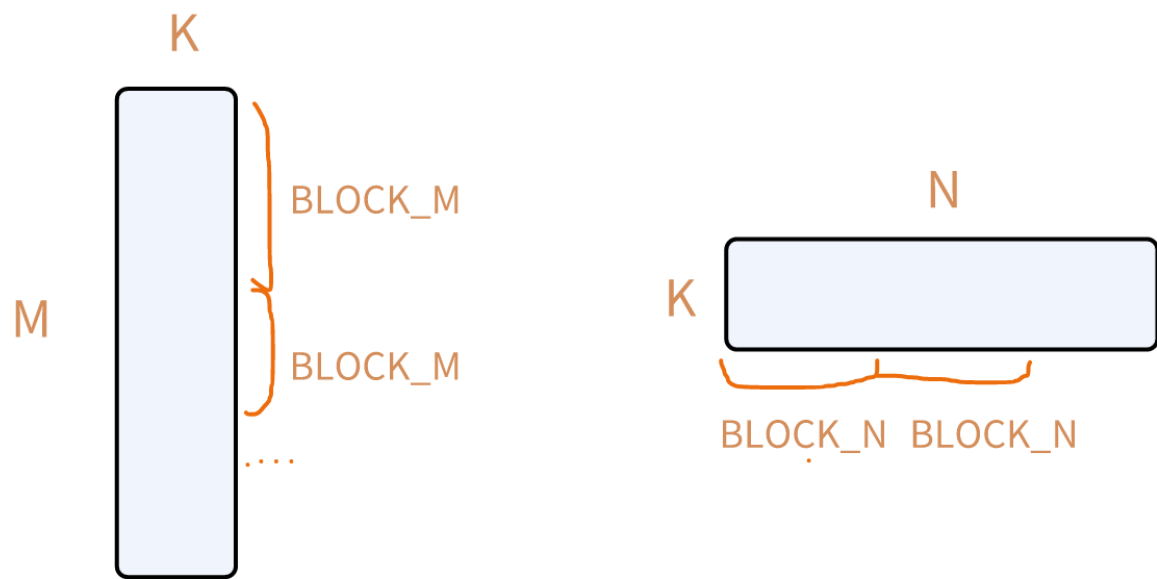
本节课的内容见https://github.com/zjhelloworld/triton_course中的对应目录。

回顾

我们在上节中用Triton实现了分块矩阵，并且在计算速度上已经和Pytorch原生的矩阵乘法处于同一个速度数量级上，让我们回顾一下它的实现原理：

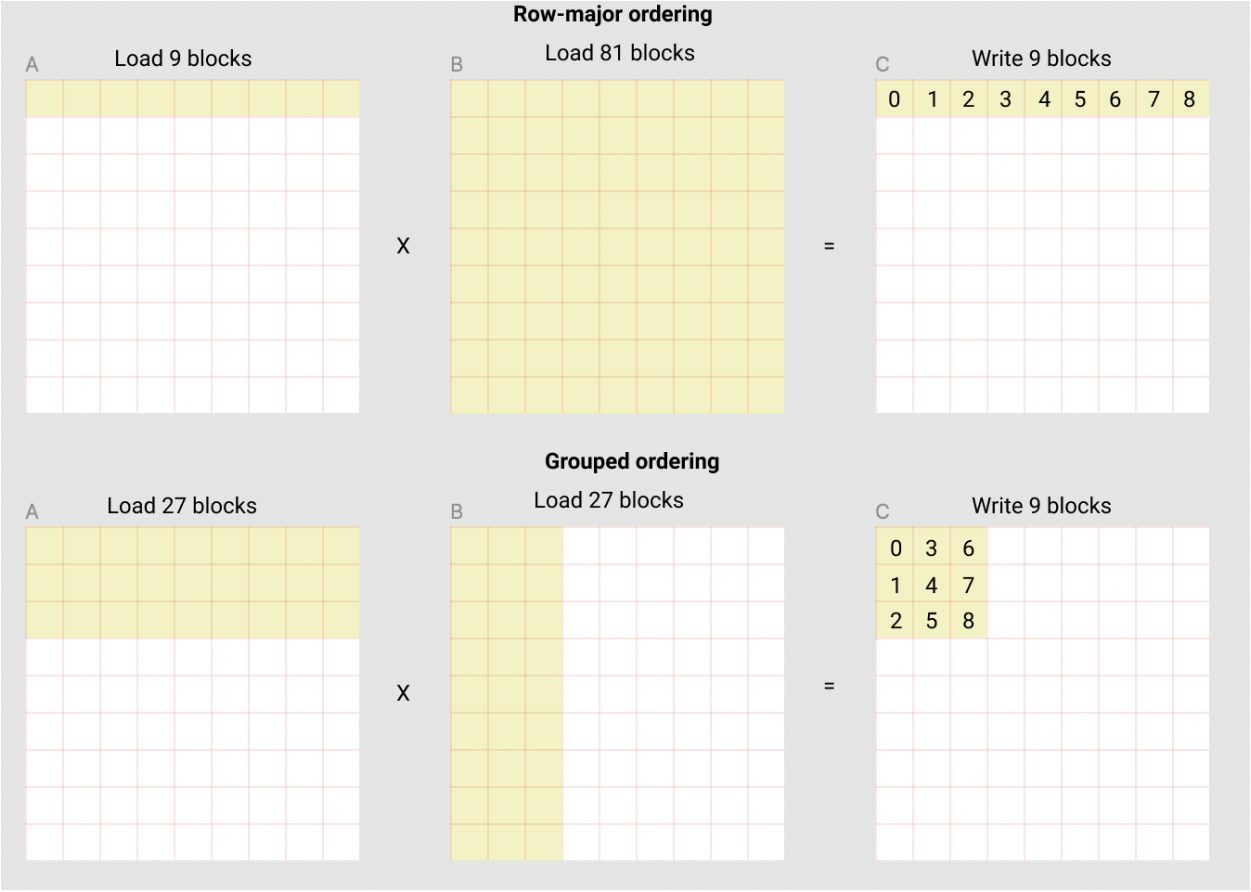
在Triton的内核实现中，每个块（block）的编号为 pid_m 和 pid_n，分别对应矩阵的 M 轴和 N 轴。因此，**每个块**负责计算的分块矩阵的维度范围可以表示为：

- M 轴范围： $[pid_m \times BLOCK_M, pid_m \times BLOCK_M + BLOCK_M)$ ，我们记作 **offs_m**。
- N 轴范围： $[pid_n \times BLOCK_N, pid_n \times BLOCK_N + BLOCK_N)$ ，我们记作 **offs_n**。
- 每个分块的计算结果为 $BLOCK_M \times BLOCK_N$ ，我们记作 **z**。

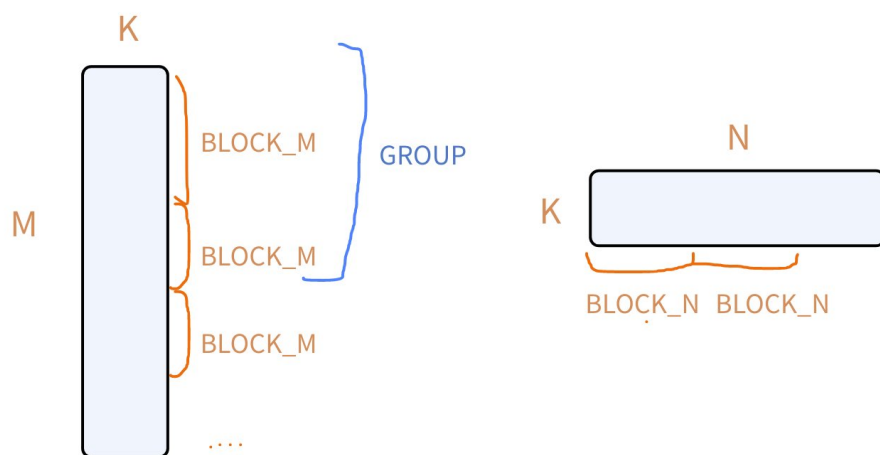


如上图所示的这种分块方式，使得每个块可以独立地处理其负责的矩阵分块，最后的结果就是这些矩阵分块的拼接，从而提高了计算的并行性和缓存利用率。我们先来分析一下现有的分块矩阵对矩阵乘法的优化，下图中每个黄色小块的大小是 `BLOCK_SIZE_M x BLOCK_SIZE_N` ,也就是一个 `block` ，图中分别展示了矩阵乘法 kernel 的两种循环方式。

我们现有的优化方式是一次加载多个行，以下图为例就是一次性加载3行，得到大小分别为(3, K) 和 (K, 3) 的子矩阵，其中K是A矩阵的列，B矩阵的行。两个子矩阵相乘得到的结果大小为 (3 × 3) 个 `block` ，而每次需要仅需要加载 $3K + 3K = 6K = 54$ 个 `block` ，此处K等于9。而上面一种，一次性加载的子矩阵分别是 (1, K) 和 (K, 9) 得到的结果为(1 × 9) 个 `block` ，而一次乘法需要加载 $K + 9K$ 个 `block` 。



本节课中，我们将更进一步将多个相邻的线程块编成一个组，相邻线程块处理的数据更有可能在缓存中命中，从而减少重复加载数据的开销。



此外，我们引入分组的概念：在同一个组…



lmhuang 2月17日 19:59

病句，让人误解

分组矩阵乘

分组信息

基于上述思路，我们进一步实现分块矩阵的优化——分组矩阵。通过将线程块分组，可以确保同一组内的线程块访问相邻的内存区域。这种局部性访问模式能够更好地利用缓存，减少内存访问延迟，从而提高计算效率。

1. 假设在 **M轴** 上共有 $M / \text{BLOCK_SIZE_M}$ 个线程块，记作 num_pid_m ，每个线程块负责处理大小为 BLOCK_SIZE_M 的数据；在 **N轴** 上同理，共有 $N / \text{BLOCK_SIZE_N}$ 个线程块，记作 num_pid_n ，每个线程块负责处理大小为 BLOCK_SIZE_N 的数据。
2. 此外，我们引入分组的概念：在同一个组内，**M轴** 上有 GROUP_SIZE_M 个线程块。例如，在上图中 $\text{GROUP_SIZE_M} = 2$ ，那么每个组内共有 $\text{GROUP_SIZE_M} \times \text{num_pid_n}$ 个线程块，我们记作 num_pid_in_group 。这种分组方式旨在优化内存访问的局部性、提高缓存命中率，并实现更均衡的负载分配。这里就符合一个组中具有多个block的设定。

代码块

```
1 pid = tl.program_id(axis=0)
2 num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
3 num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
4 num_pid_in_group = GROUP_SIZE_M * num_pid_n
```

确定偏移

在确定组的大小以及每个组包含的线程块数量后，我们需要定位当前线程块所在的组，以及它在 **M轴** 和 **N轴** 上的具体位置。

1. 已知每个组内的线程块数量为 num_pid_in_group ，因此当前线程块所属的组 ID 可以通过 $\text{group_id} = \text{pid} // \text{num_pid_in_group}$ 计算得到。该组在 **M轴** 上的起始位置为组 ID 乘以每个组在 **M轴** 上的线程块数量，即 $\text{first_pid_m} = \text{group_id} \times \text{GROUP_SIZE_M}$ 。
2. 对于当前线程块 pid ，它在 **M轴** 上的位置可以通过以下公式计算：
 $\text{pid_m} = \text{first_pid_m} + (\text{pid} \% \text{group_size_m})$ ，其中 group_size_m 是当前组在 **M轴** 上的实际线程块数量，它需要比一个组内在M轴的最大线程块 GROUP_SIZE_M 小。
3. 在 **N轴** 上的位置则通过以下方式确定： $\text{pid_n} = (\text{pid} \% \text{num_pid_in_group}) // \text{group_size_m}$ ，即当前线程块在组内的偏移量除以组内在 **M轴** 上的线程块数量。

简单理解以上的内容：

1. 首先确定每个组包含的线程块总数， num_pid_in_group ，首先是线程块所属的组id：

$\text{first_pid_m} + \text{pid} \% \text{group_size_m}$



Yue 2025年12月31日

```
1 group_id = pid // num_pid_in_group
```

2. 随后确定该组在M轴上的起始位置：

代码块

```
1 first_pid_m = group_id × GROUP_SIZE_M
```

为了同学们能更好地理解，我画了一个示意图，其中：

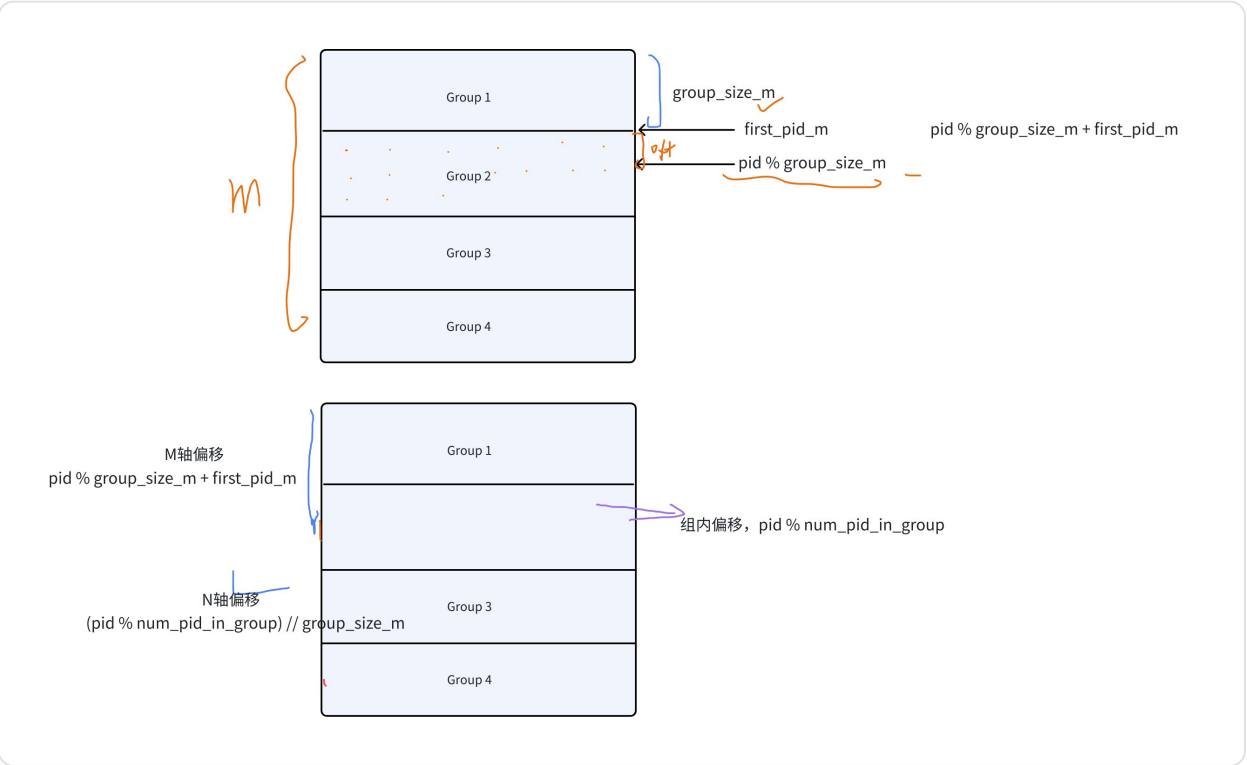
- 1. first_pid_m是对于当前线程块所属组在M轴上的起始位置。
- 2. group_size_m是一个组在M轴上有多少个线程块。因此当前线程块pid在M轴上的偏移就有first_pid_m+pid % group_size_m
- 3. 在N轴上的偏移我们先线程块在计算组内的偏移，也就是pid % num_pid_in_group，最后再计算N轴的坐标，也就是将组内的偏移除以group_size_m（一个组在M轴上有多少个线程块），即(pid % num_pid_in_group) // group_size_m

在m轴和n轴上的偏移

代码块

```
1 group_id = pid // num_pid_in_group
2 first_pid_m = group_id * GROUP_SIZE_M # 所在的组开始的m轴坐标
3 group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)
4 pid_m = first_pid_m + (pid % group_size_m) # m轴的坐标
5 pid_n = (pid % num_pid_in_group) // group_size_m
```

下面的图示请结合视频一起看，最后是为了得到pid_m和pid_n这两个块的索引。



在确定了分组大小和线程块pid的组号、偏移后，我们就需要在规约轴K轴上进行规约，这部分代码和我们分块矩阵乘法的实现基本是一致的。

a_ptrs和b_ptrs分别表示当前线程块处理的a子矩阵和b子矩阵的偏移。在k轴上进行规约时，每次移动的位置为BLOCK_SIZE_K×stride_ak或者BLOCK_SIZE_K×stride_bk。其中对A、B矩阵的索引坐标为：

- 1. offs_am[:, None] * stride_am + offs_k[None, :] * stride_ak 对应的A矩阵的二维坐标范围是 (offs_am, offs_k)

这里公式是不是写错了，应该是代码里的 pid_m = first_pid_m + ((pid % num_pid_in_group) % group_size_m)

ArdenX 3月10日 00:16 @Yue <https://zhuanlan.zhihu.com/p/684473453> 感觉他看的是这一篇，但有些没写清楚

ArdenX 3月10日 00:38 @Yue 组内是列主序

pid_m = first_pid_m + (pid % group_size_m)

Yue 2025年12月31日 pid_m = first_pid_m + ((pid % num_pid_in_group) % group_size_m)

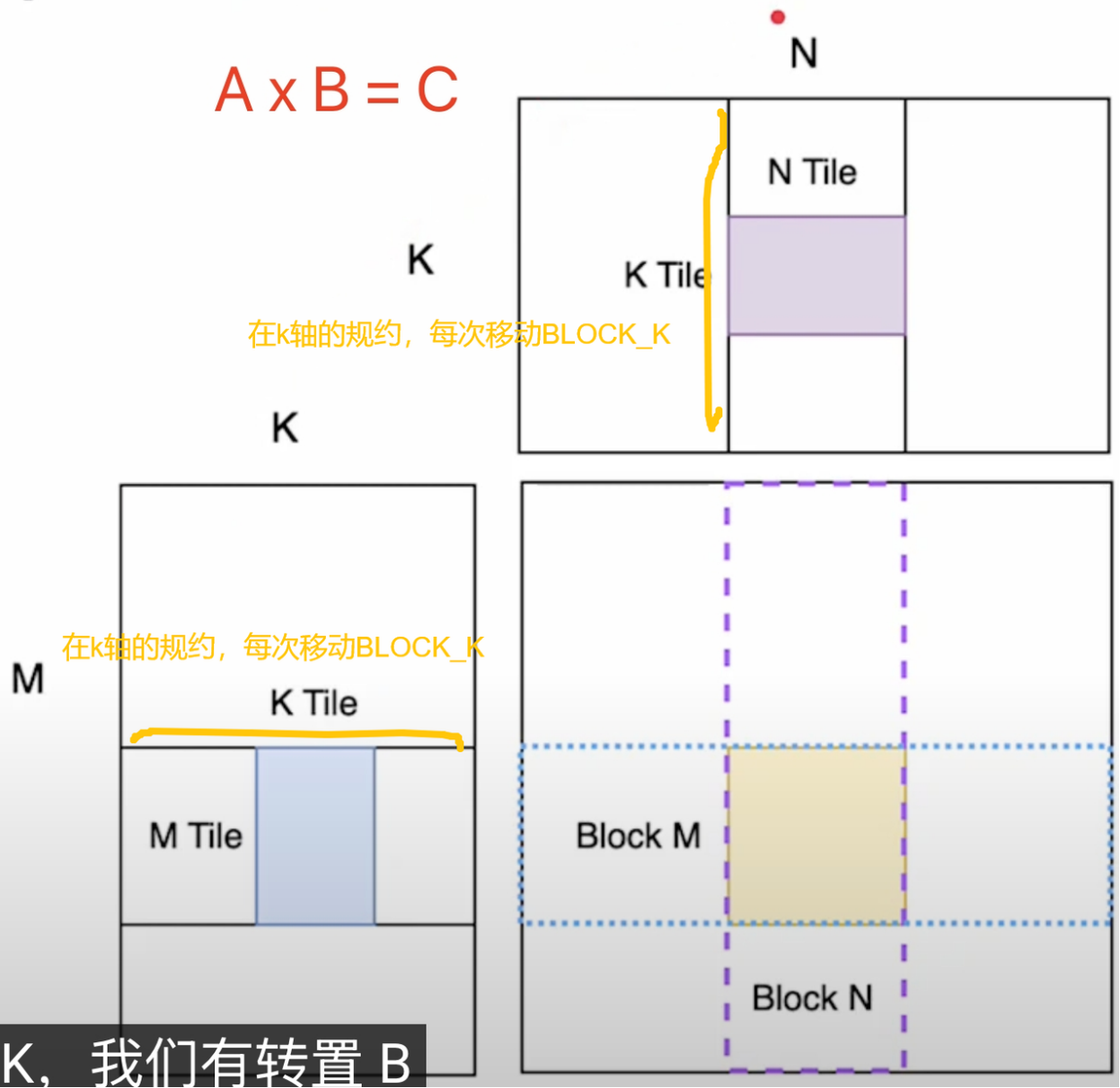
2. 同理offs_k[:, None] * stride_bk + offs_bn[None, :] * stride_bn对应的B矩阵的二维坐标范围是(offs_k, offs_bn)

这里唯一和第3节不同的是索引的方式不同。

代码块

```
1  offs_am = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)) % M
2  offs_bn = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)) % N
3  offs_k = tl.arange(0, BLOCK_SIZE_K)
4  a_ptrs = a_ptr + (offs_am[:, None] * stride_am + offs_k[None, :] *
5  stride_ak)
6
7  b_ptrs = b_ptr + (offs_k[:, None] * stride_bk + offs_bn[None, :] *
8  stride_bn)
9
10 accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N),
11 dtype=tl.float32)
12 for k in range(0, tl.cdiv(K, BLOCK_SIZE_K)):
13     a = tl.load(a_ptrs, mask=offs_k[None, :] < K - k *
14     BLOCK_SIZE_K, other=0.0)
15     b = tl.load(b_ptrs, mask=offs_k[:, None] < K - k *
16     BLOCK_SIZE_K, other=0.0)
17     accumulator += tl.dot(a, b)
18     a_ptrs += BLOCK_SIZE_K * stride_ak
19     b_ptrs += BLOCK_SIZE_K * stride_bk
20 c = accumulator.to(tl.float16)
```

如下图所示，在k轴上进行规约时，A子矩阵和B子矩阵每次在规约轴上移动的大小范围为BLOCK_K。



核函数的启动配置

```

1  c = torch.empty((M, N), device=a.device, dtype=a.dtype)
2  grid = lambda META: (
3      triton.cdiv(M, META['BLOCK_SIZE_M']) * triton.cdiv(N,
4      META['BLOCK_SIZE_N']),
5      )
6  matmul_kernel[grid](
7      a, b, c,
8      M, N, K,
9      a.stride(0), a.stride(1),
10     b.stride(0), b.stride(1),
11     c.stride(0), c.stride(1), 32, 32, 32, 4,
12 )

```

按照上文的讲解，我们在网格中分别需要为M轴和N轴分别配置M/BLOCK_SIZE_M和N/BLOCK_SIZE_N个线程块，同时还需要为矩阵乘法的结果准备输出空间c。同时我们在M, N和K轴上的分块大小均为32，也就是一个线程块需要处理的维度大小是32，同时将4个线程块归为一组（group = 4）。

结果和速度的对比

我们先给出完整的kernel代码以及对应的核函数配置情况。BLOCK_SIZE_K，BLOCK_SIZE_M和BLOCK_SIZE_N是一个可以配置的参数，autotune会去选择最优的参数进行保存。

代码块

```

1  @triton.autotune(
2      configs=get_autotune_config(),
3      key=['M', 'N', 'K'],
4  )
5  @triton.jit
6  def matmul_kernel(
7      a_ptr, b_ptr, c_ptr,
8      M, N, K,
9      stride_am, stride_ak,
10     stride_bk, stride_bn,
11     stride_cm, stride_cn,
12     BLOCK_SIZE_M: tl.constexpr, BLOCK_SIZE_N: tl.constexpr,
13     BLOCK_SIZE_K: tl.constexpr,
14     GROUP_SIZE_M: tl.constexpr,
15 ):
16     """Kernel for computing the matmul C = A x B.
17     A has shape (M, K), B has shape (K, N) and C has shape (M, N)
18     """
19     pid = tl.program_id(axis=0)
20     num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
21     num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
22     num_pid_in_group = GROUP_SIZE_M * num_pid_n
23     group_id = pid // num_pid_in_group
24     first_pid_m = group_id * GROUP_SIZE_M
25     group_size_m = min(num_pid_m - first_pid_m, GROUP_SIZE_M)
26     pid_m = first_pid_m + (pid % group_size_m)
27     pid_n = (pid % num_pid_in_group) // group_size_m
28
29     offs_am = (pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M))
30     % M
31     offs_bn = (pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N))
32     % N
33     offs_k = tl.arange(0, BLOCK_SIZE_K)

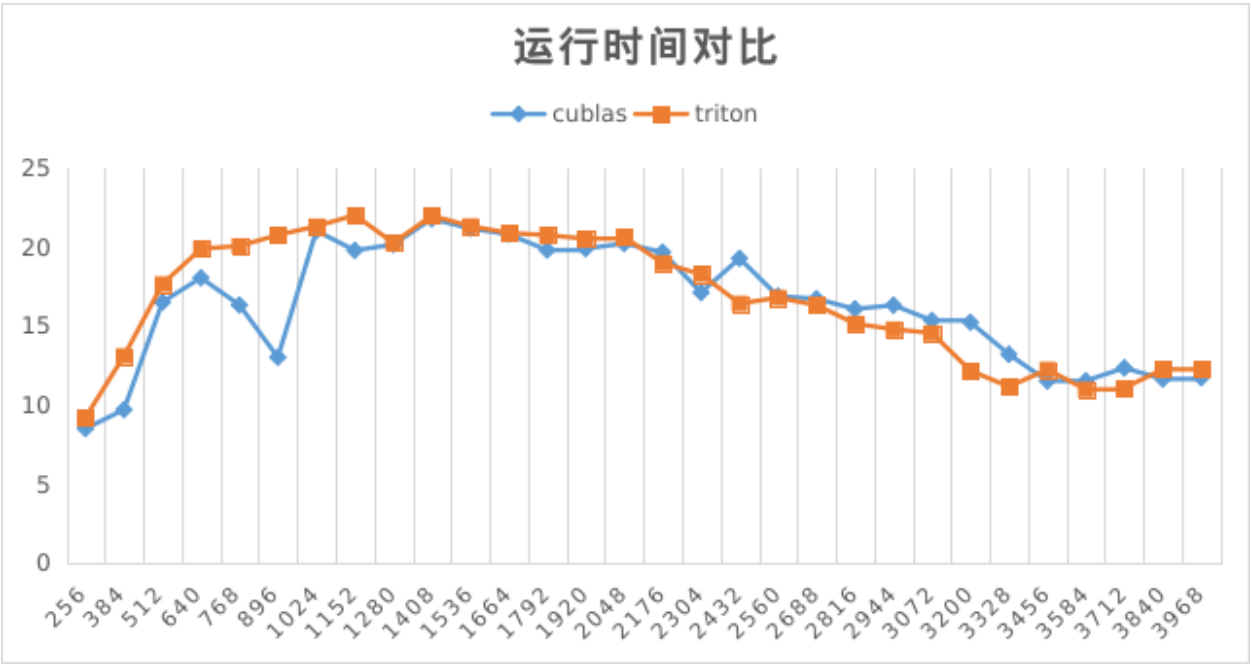
```

```

32     a_ptrs = a_ptr + (offs_am[:, None] * stride_am + offs_k[None,
:] * stride_ak)
33     b_ptrs = b_ptr + (offs_k[:, None] * stride_bk + offs_bn[None,
:] * stride_bn)
34
35     accumulator = tl.zeros((BLOCK_SIZE_M, BLOCK_SIZE_N),
dtype=tl.float32)
36     for k in range(0, tl.cdiv(K, BLOCK_SIZE_K)):
37         a = tl.load(a_ptrs, mask=offs_k[None, :] < K - k *
BLOCK_SIZE_K, other=0.0)
38         b = tl.load(b_ptrs, mask=offs_k[:, None] < K - k *
BLOCK_SIZE_K, other=0.0)
39         accumulator += tl.dot(a, b)
40         a_ptrs += BLOCK_SIZE_K * stride_ak
41         b_ptrs += BLOCK_SIZE_K * stride_bk
42     c = accumulator.to(tl.float16)
43
44     offs_cm = pid_m * BLOCK_SIZE_M + tl.arange(0, BLOCK_SIZE_M)
45     offs_cn = pid_n * BLOCK_SIZE_N + tl.arange(0, BLOCK_SIZE_N)
46     c_ptrs = c_ptr + stride_cm * offs_cm[:, None] + stride_cn *
offs_cn[None, :]
47     c_mask = (offs_cm[:, None] < M) & (offs_cn[None, :] < N)
48     tl.store(c_ptrs, c, mask=c_mask)
49
50 def matmul(a, b):
51     a = a.view((-1, a.shape[-1]))
52     assert a.shape[1] == b.shape[0], "Incompatible dimensions"
53     assert a.is_contiguous(), "Matrix A must be contiguous"
54     assert b.is_contiguous(), "Matrix B must be contiguous"
55     M, K = a.shape
56     K, N = b.shape
57     c = torch.empty((M, N), device=a.device, dtype=a.dtype)
58     grid = lambda META: (
59         triton.cdiv(M, META['BLOCK_SIZE_M']) * triton.cdiv(N,
META['BLOCK_SIZE_N']),
60     )
61     matmul_kernel[grid](
62         a, b, c,
63         M, N, K,
64         a.stride(0), a.stride(1),
65         b.stride(0), b.stride(1),
66         c.stride(0), c.stride(1), 32, 32, 32, 4,
67     )
68     return c

```

通过比较Triton矩阵乘的输出结果output与Pytorch矩阵乘的结果，我们发现两者完全一致（差异极小）。另外，我们也将测试Triton Kernel和Pytorch矩阵乘的速度差异，可以看到我们实现的算子还是和Pytorch有一定的速度差异，但是已经在同一个数量级上。我们在验证时，矩阵乘法使用的输入维度从128到4096，每次增长间隔128。

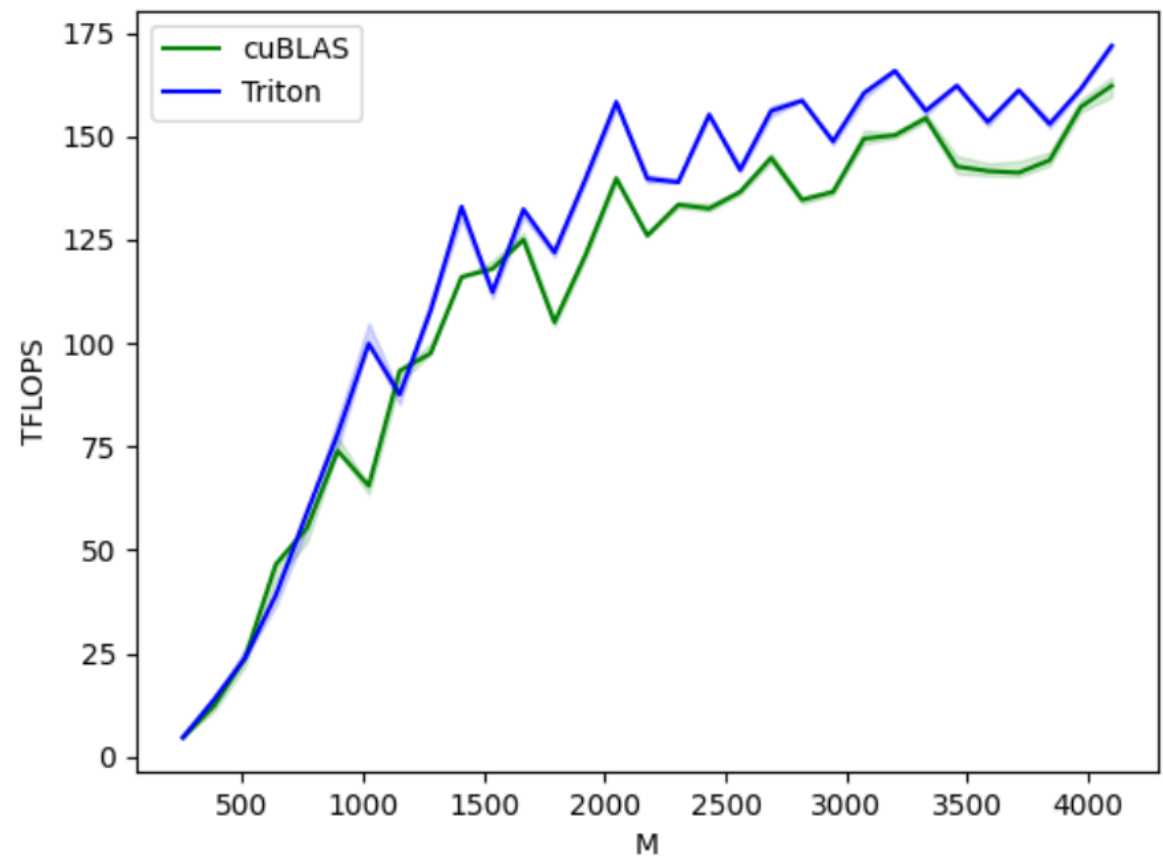


从图中能够清晰地观察到，经过分组处理后的矩阵乘法在运算速度上实现了进一步提升，甚至超过cuBlas，不过在特定的维度范围上仍存在一些细微的差距，图表x轴是矩阵的维度大小，y轴为各自算子的运行时间。所以经过我们不算太复杂的调优流程，triton总体就可以达到和cublas不相上下的速度。

参数调优

我们认为，尽管 Triton 作为一款强大的自动调优工具，在优化方面有着自身的独特之处，但其优化深度或许存在一定的局限性。Triton 主要依赖自动搜索参数空间的方式来确定最优配置，然而，相较于人工优化而言，它难以充分挖掘并利用硬件的所有潜在特性。现在我们来分析上一节中用到的算子参数自动调优技术，我们在注解 `triton.autotune` 中调用了 `get_autotune_config()`，该函数返回了一组参数用于为 triton 的核函数选择最佳参数，包括在 M, N, K 轴上的分块大小，`num_warps` 的大小（也就是 cuda 中一个 block 的大小）以及其他的一些可选参数。

在 `get_cuda_autotune_config` 中，我们为 M, N, K 轴上的分块大小分别配置了 16, 32, 64, 128, 256，`num_warps` 的大小配置了 2, 4, 8，`num_stages` 的大小配置了 3, 4, 5。当开始执行之前，kernel 会在参数空间中进行搜索，找到最优的参数，然后以这些最优参数执行。keys=['M', 'N', 'K'] 表示 kernel 的输入维度，当 M, N, K 的值发生变化时，kernel 会重新进行搜索。可以看到经过调优之后的，triton 速度和 cublas 相比更有一些优势。



triton速度和cublas相比更有一些优势。



汤毅辰 2025年11月23日

感觉比分块有提升但不大

